Cheat Sheet & PPT Content

NumPy for Numerical Computing and the Transition into Pandas and Machine Learning

1. NUMPY BASICS

```
import numpy as np
```

- NumPy provides fast, efficient and memory-friendly operations on arrays.
- Used for numerical computing, data analysis, ML and scientific computing.

Creating an array

```
a = np.array([1, 2, 3])
print(a)  # [1 2 3]
```

Array dimensions

```
1D:  a = np.array([1, 2, 3])           # (3,)
2D:  b = np.array([[1, 2], [3, 4]])    # (2, 2)
3D:  c = np.array([[[1, 2], [3, 4]],
                   [[5, 6], [7, 8]]])  # (2, 2, 2)
```

Key attributes

```
a.ndim   # number of dimensions
a.shape  # shape of array
a.size   # total number of elements
a.dtype  # data type
```

Example

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

a.ndim   # 2
a.shape  # (2, 3)
a.size   # 6
a.dtype  # dtype('int64')
```

2. CREATING ARRAYS

Function	Syntax	Example	Use
zeros()	np.zeros(shape)	np.zeros((2, 3))	Fill with 0
ones()	np.ones(shape)	np.ones((2, 3))	Fill with 1
full()	np.full(shape, val)	np.full((2, 3), 7)	Fill with value
arange()	np.arange(start, stop, step)	np.arange(0, 10, 2)	Step-based sequence
linspace()	np.linspace(start, stop, num)	np.linspace(0, 1, 5)	Fixed number of points

Examples

```
np.zeros((2, 3))        # [[0. 0. 0.]
                        #  [0. 0. 0.]]
np.ones((2, 3))         # [[1. 1. 1.]
                        #  [1. 1. 1.]]
np.full((2, 3), 7)      # [[7 7 7]
                        #  [7 7 7]]
np.arange(0, 10, 2)     # [0 2 4 6 8]
np.linspace(0, 1, 5)    # [0.  0.25 0.5 0.75 1.]
```

3. INDEXING & SLICING

Accessing elements

```
a = np.array([[10, 20, 30],
              [40, 50, 60]])

a[2]     # 3rd element (1D)
a[1:4]   # elements 1 to 3
a[1, 2]  # element at row 1, col 2
a[:, 1]  # all rows, col 1
a[1, :]  # row 1, all columns
```

Modifying elements

```
a[1, 2] = 99
print(a)   # [[10 20 30]
           #  [40 99 60]]
```

Example array (2D)

	0	1	2
0	10	20	30
1	40	50	60

Common indexing patterns

Row 0	a[0, :]
Column 1	a[:, 1]
Submatrix	a[0:2, 1:3]

4. ELEMENT-WISE OPERATIONS & UNIVERSAL FUNCTIONS

Element-wise arithmetic

```
a + b    # addition
a - b    # subtraction
a * b    # multiplication
a / b    # division
a ** 2   # power
```

Comparison operators

```
a > 5    # greater than
a == 3   # equal to
a != 3   # not equal to
(a > 2) & (a < 8)   # and
```

Universal functions (ufuncs)

```
np.sqrt(a)  # square root
np.abs(a)   # absolute value
np.exp(a)   # exponential
np.log(a)   # natural log
np.sin(a)   # sine
np.cos(a)   # cosine
```

Example

```
a = np.array([1, 4, 9])
np.sqrt(a)   # [1. 2. 3.]
```

5. AGGREGATION & AXIS

Common aggregation functions

```
np.sum()   # sum of elements
np.mean()  # mean
np.min()   # minimum
np.max()   # maximum
np.std()   # standard deviation
```

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

np.sum(a)   # 21
np.mean(a)  # 3.5
```

Using axis

```
np.sum(a, axis=0)  # column-wise
np.sum(a, axis=1)  # row-wise
```

Axis explanation (for 2×3 array)

1	2	3
4	5	6

axis=0 (columns) runs **down** the rows;
axis=1 (rows) runs **across** the columns.

```
np.sum(a, axis=0)  # [5 7 9]
np.sum(a, axis=1)  # [ 6 15]
```

6. RESHAPING, FLATTEN & TRANSPOSE

Reshaping

```
a = np.arange(6)   # [0 1 2 3 4 5]
a.reshape(2, 3)   # [[0 1 2]
                  #  [3 4 5]]
```

Flatten vs Ravel

```
a.flatten()  # returns a copy
a.ravel()    # returns a view (if possible)
```

Transpose

```
a.T   # swaps axes (rows ↔ columns)
```

Example (2×3 array)

```
a = np.array([[1, 2, 3],
              [4, 5, 6]])

a.reshape(3, 2)  # [[1 2]
                 #  [3 4]
                 #  [5 6]]
a.flatten()      # [1 2 3 4 5 6]
a.ravel()        # [1 2 3 4 5 6]
a.T              # [[1 4]
                 #  [2 5]
                 #  [3 6]]
```

NumPy CHEAT SHEET — PAGE 2

Array Manipulation & Data Selection

Combine → Split → Filter → Select
Broadcast → Copy → Convert →
Sort/Search

1. CONCATENATION & STACKING

Join multiple arrays together.

np.concatenate() – join along an existing axis

```
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6], [7, 8]])

np.concatenate([a, b], axis=0)  # vertical
# [[1 2]
#  [3 4]
#  [5 6]
#  [7 8]]

np.concatenate([a, b], axis=1)  # horizontal
# [[1 2 5 6]
#  [3 4 7 8]]
```

vstack(), hstack()

```
np.vstack([a, b])  # vertical
np.hstack([a, b])  # horizontal
```

stack() – creates a new axis

```
np.stack([a, b], axis=0)
# shape: (2, 2, 2)
```

Example (a and b)

	Result	Shape
vstack	[[1 2] [3 4] [5 6] [7 8]]	(4, 2)
hstack	[[1 2 5 6] [3 4 7 8]]	(2, 4)
stack (new axis)	[[[1 2] [3 4]] [[5 6] [7 8]]]	(2, 2, 2)

Quick comparison

Function	Idea
concatenate()	Join along existing axis
vstack()	Vertical stacking
hstack()	Horizontal stacking
stack()	Join + create new axis

Note: Arrays must have compatible shapes along the other axes (except for the stacking axis).

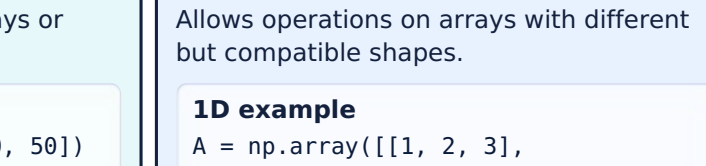
2. SPLITTING ARRAYS

Break one array into multiple smaller arrays.

```
a = np.arange(8)
np.split(a, 2)        # equal parts
np.array_split(a, 3)  # unequal parts
np.vsplit(a, 2)       # split vertically
np.hsplit(a, 2)       # split horizontally
```

Example

```
a = np.arange(8)
np.split(a, 2)
# [array([0, 1, 2, 3]), array([4, 5, 6, 7])]
```



- **split()** → equal divisions required
- **array_split()** → unequal divisions allowed

3. BOOLEAN MASKING ★

Select elements using a condition (returns a boolean mask).

```
a = np.array([2, 7, 4, 9, 1])

a > 5      # [False  True False  True False]
a[a > 5]   # [7 9]
```

Common patterns

```
a[a == 4]   # equal to
a[a != 4]   # not equal to
a[a >= 5]   # greater or equal
a[a < 5]    # less than
```

Multiple conditions

```
(a > 2) & (a < 9)    # AND
(a == 2) | (a == 9)  # OR
~(a == 2)            # NOT

a[(a > 2) & (a < 9)]
```

Use **&** / **|** instead of Python's **and** / **or**.

Create and reuse a mask

```
mask = a > 5
a[mask]
```

2D example

```
A = np.array([[5, 8, 1],
              [2, 6, 3],
              [7, 4, 9]])

A[A > 5]  # [8 6 7 9] (flattened)
```

How it works

```
A     = [2 7 4 9 1]
A > 5 = [F T F T F]

Selected: [7 9]
```

Key idea: condition → True/False mask → selected values

4. FANCY / INTEGER INDEXING ★

Select elements using index arrays or lists.

1D example

```
a = np.array([10, 20, 30, 40, 50])

a[[0, 2, 4]]   # [10 30 50]
```

2D examples

```
A = np.array([[10, 20, 30],
              [40, 50, 60],
              [70, 80, 90]])

A[[0, 2]]        # select rows 0 and 2
# [[10 20 30]
#  [70 80 90]]

A[[0, 2], [1, 2]]  # A[0,1] and A[2,2]
# [20 90]
```

Boolean vs Fancy indexing

Boolean indexing	Fancy indexing
condition-based	index-based
a[a > 5]	a[[0, 2, 4]]

Remember:
Boolean → "Which values satisfy this?"
Fancy → "Give me these positions."

5. BROADCASTING ★★★

Allows operations on arrays with different but compatible shapes.

1D example

```
A = np.array([[1, 2, 3],
              [4, 5, 6]])
b = np.array([10, 20, 30])

A + b
# [[11 22 33]
#  [14 25 36]]
```

A (2, 3)	+	b (3,)	→	Result (2, 3)
[[1 2 3] [4 5 6]]	+	[10 20 30]	→	[[11 22 33] [14 25 36]]

Broadcasting rule

Compare dimensions from right to left.
Two dimensions are compatible when:

- they are equal, or
- one of them is 1.

Examples

(2, 3)	(3,)	✓
(2, 3)	(2, 1)	✓
(2, 3)	(2, 2)	✗

Scalar broadcasting

```
A + 10
A * 2   # applied to every element
```

Key idea: Broadcasting avoids manually copying/repeating data.

6. COPIES VS VIEWS ★

How data is shared between arrays.

Copy (independent data)

```
b = a.copy()
b[0] = 99
# a remains unchanged
```

View (shared data)

```
b = a.view()
b[0] = 99
# a may also change
```

Quick comparison

Feature	Copy	View
New array object	✓	✓
Shares data	✗	✓
Changes affect original	✗	Usually ✓
Syntax	a.copy()	a.view()

Memory view

```
Copy:  A → [data]
       B → [different data]

View:  A → [same data]
       B → [same data]
```

Note: b = a is neither a copy nor a view — it's just another reference to the same array.

7. DATA TYPES & ASTYPE()

Each array has a data type (dtype).

Check dtype

```
a = np.array([1, 2, 3])
a.dtype   # int64 (example)
```

Common types

- **int** – integers
- **float** – floating point
- **bool** – boolean
- **complex** – complex numbers

Convert dtype

```
a.astype(float)  # [1. 2. 3.]
a.astype(int)    # [1 2 3]
a.astype(bool)   # [ True  True True]
```

Note: Converting to int removes the fractional part (does not round).

Why it matters?

- Memory usage
- Numerical precision
- Computation performance
- Library compatibility

8. SORTING & SEARCHING

Organize data and find positions/values.

Sorting

```
a = np.array([40, 10, 30, 20])
np.sort(a)   # [10 20 30 40]
→ returns a sorted copy
```

argsort(a)

```
np.argsort(a)  # [1 3 2 0]
→ indices that would sort the array
```

Minimum / maximum position

```
np.argmin(a)   # index of smallest
np.argmax(a)   # index of largest
```

where()

```
a = np.array([10, 30, 20, 40])
np.where(a > 25)   # (array([1, 3]),)
```

Quick reference

Function	Purpose
np.sort(a)	Sorted values
np.argsort(a)	Sorting indices
np.argmin(a)	Index of minimum
np.argmax(a)	Index of maximum
np.where(cond)	Positions satisfying condition

QUICK REFERENCE (PAGE 2)

COMBINE
- concatenate()
- vstack()
- hstack()
- stack()

SPLIT
- split()
- array_split()
- vsplit()
- hsplit()

SELECT
- Boolean: a[a > x]
- Fancy: a[[0, 2, 4]]

BROADCAST
- Compatible shapes → equal OR 1

COPY
- copy() → independent
- view() → shared data

DTYPE
- dtype → inspect
- astype() → convert

SEARCH
- sort()
- argsort()
- argmin()
- argmax()
- where()

NumPy CHEAT SHEET — PAGE 3

Numerical Computing → Machine Learning

Math & Stats → Random → Linear Algebra → ML Patterns → NumPy → Pandas → ML

1. MATHEMATICAL & STATISTICAL OPERATIONS

Perform common mathematical, statistical and aggregate operations on arrays.

Mathematical operations

```
a = np.array([1, 2, 3, 4])

np.sqrt(a)      # square root
np.exp(a)       # exponential
np.log(a)       # natural log
np.power(a, 2)  # element-wise square
np.abs(a)       # absolute value
```

Statistical operations

```
data = np.array([[1, 2, 3, 4, 5]])

np.mean(data)    # average
np.median(data)  # median
np.std(data)     # standard deviation
np.var(data)     # variance
np.min(data)     # minimum
np.max(data)     # maximum
np.sum(data)     # sum
```

Output (examples)

```
sqrt:  [1.  1.414  1.732  2.]
exp:   [2.718  7.389  20.086  54.598]
log:   [0.  0.693  1.099  1.386]
```

Example output

```
mean: 3.0   median: 3.0   std: 1.414
var: 2.0    min: 1   max: 5   sum: 15
```

2. RANDOM NUMBERS & SAMPLING

Generate and work with random numbers for simulations, sampling and testing.

Random number generation

```
np.random.seed(42)         # reproducible results
np.random.rand(3)          # [0, 1) uniform
np.random.randn(3)         # normal (mean=0, std=1)
np.random.randint(1, 10, 3)  # integers [1, 9]
np.random.choice(5, 3)     # random choice from range
```

Example output

```
rand:    [ 0.374  0.951  0.732]
randn:   [ 0.496 -0.138  0.648]
randint: [7 3 1]
choice:  [0 4 2]
```

Sampling from data

```
data = np.arange(10)

np.random.choice(data, 4)                    # without replacement
np.random.choice(data, 4, replace=True)    # with replacement
```

Example

```
Without:  [8 1 6 4]
With:     [3 3 7 2]
```

Shuffling

```
arr = np.arange(5)
np.random.shuffle(arr)
# arr is modified in place
```

3. LINEAR ALGEBRA

Work with matrices and vectors using NumPy's linear algebra functions.

Basic operations

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])

np.dot(A, B)          # matrix multiplication
A @ B                 # same as np.dot

np.transpose(A)       # transpose
np.linalg.inv(A)      # inverse
np.linalg.det(A)      # determinant
np.linalg.trace(A)    # trace
```

Example output (selected)

```
A @ B =  [19 22]
         [43 50]

transposed A =  [1 3]
                [2 4]

trace(A) = 5.0
```

Solving linear systems

```
A = np.array([[3, 1], [1, 2]])
b = np.array([9, 8])

x = np.linalg.solve(A, b)
# x = [2. 3.]
```

Eigenvalues & eigenvectors

```
A = np.array([[2, 0], [0, 3]])
vals, vecs = np.linalg.eig(A)

vals   # [2. 3.]
vecs   # [[1. 0.]
       #  [0. 1.]]
```

Key idea: Linear algebra is the backbone of many ML algorithms (e.g., regression, PCA, neural networks).

4. PRACTICAL ML PATTERNS

Common NumPy patterns used in machine learning workflows.

Feature scaling (Normalization)

```
x = np.array([1, 2, 3, 4, 5], dtype=float)

x_norm = (x - x.min()) / (x.max() - x.min())

# Min-Max scaling → range [0, 1]
```

Output:

```
x_norm: [0.  0.25  0.5  0.75  1.]
```

Train-test split

```
from sklearn.model_selection import train_test_split

X = np.arange(100).reshape(50, 2)
y = np.arange(50)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
```

(Requires scikit-learn)

Batch processing

```
X = np.random.rand(100, 10)
batch_size = 16

for i in range(0, len(X), batch_size):
    batch = X[i:i+batch_size]
    # train on batch
```

Vectorized operations (faster than loops)

```
# Instead of a loop
result = x * 2 + 1

# Works on entire array at once
```

Common ML tasks with NumPy

- Data preprocessing (scaling, normalization, encoding)
- Feature engineering (polynomials, interactions)
- Metrics (accuracy, loss calculation)
- Model operations (matrix multiplication, gradients)

5. NUMPY → PANDAS → ML (THE BIG PICTURE)

NumPy handles raw numerical data. Pandas adds labeled data & easy analysis.
ML libraries (like scikit-learn) use NumPy/Pandas under the hood.

NumPy (ndarrays)	Pandas (DataFrames)	ML (scikit-learn)
• fast • memory efficient • numerical computing	• labeled data • data cleaning • analysis & visualization	• preprocessing • model training • evaluation

NumPy → Pandas → ML

Example workflow

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split

# 1. Load data into NumPy array
X = np.array([[1, 2], [3, 4], [5, 6]])
y = np.array([0, 1, 0])

# 2. Convert to Pandas for easier handling
df = pd.DataFrame(X, columns=['f1', 'f2'])
df['label'] = y

# 3. Split for ML
X_train, X_test, y_train, y_test = train_test_split(
    df[['f1', 'f2']], df['label'],
    test_size=0.2, random_state=42)
```

Why use all three?	Real-world flow (simplified)
✓ **NumPy:** fast numerical operations ✓ **Pandas:** data cleaning & manipulation ✓ **scikit-learn:** ML models & tools	Raw data (CSV/JSON) ↓ NumPy (arrays) ↓ Pandas (DataFrame) ↓ Preprocessing + ML ↓ Model output

6. QUICK TIPS & USEFUL CONCEPTS

1. Avoid Python loops (use vectorization)

```
# Slower
res = [x*2 for x in arr]

# Faster
res = arr * 2
```

2. Watch out for array shapes

```
# (2, 3) + (3,)   → works (broadcast)
# (2, 3) + (2, 1) → works
# (2, 3) + (2, 2) → error
```

3. Use proper dtypes

```
# For memory efficiency
arr = np.array([1, 2, 3], dtype=np.float32)
```

4. Fix random seeds

```
np.random.seed(0)  # reproducible results
```

5. Handle missing values (in ML)

```
np.isnan(arr)       # find NaN
np.nan_to_num(arr)  # replace NaN
np.mean(arr, where=~np.isnan(arr))
```

6. Common gotchas

- a = b (reference, not copy)
- Use & / | instead of **and** / **or**
- np.matrix is deprecated
- Check array shapes before operations

Remember: NumPy is fast, powerful and flexible — but understanding shapes, dtypes and broadcasting will save you a lot of time!

QUICK REFERENCE — PAGE 3

MATH & STATS
- sqrt, exp, log, abs, power
- mean, median, std, var
- min, max, sum

RANDOM
- rand, randn, randint
- choice, shuffle
- seed()

LINEAR ALGEBRA
- dot / @, transpose
- inv, det, trace
- solve, eig

ML PATTERNS
- normalization
- train_test_split
- vectorization

NUMPY → PANDAS → ML
- ndarrays → DataFrame
- clean + preprocess
- train + evaluate

EXTRA TIPS
- vectorize
- correct dtype
- watch shapes
- set seed